Day 41 – Tree Problems (Important Set)

1. Height of Binary Tree

👉 Height = longest path from root to leaf

Formula:

$$h = 1 + \max(h_{left}, h_{right})$$

Code (Java)

```java
int height(Node root) {
    if (root == null) return 0;

    int left = height(root.left);
    int right = height(root.right);

    return 1 + Math.max(left, right);
}
```

2. Count Total Nodes

```java
int countNodes(Node root) {
    if (root == null) return 0;

    return 1 + countNodes(root.left) + countNodes(root.right);
}
```

3. Sum of All Nodes

```java
int sum(Node root) {
    if (root == null) return 0;

    return root.data + sum(root.left) + sum(root.right);
}
```

4. Check if Tree is Balanced

👉 Balanced = left height & right height difference ≤ 1

```
int height(Node root) {
    if (root == null) return 0;

    int left = height(root.left);
    int right = height(root.right);

    if (Math.abs(left - right) > 1) return -1;

    if (left == -1 || right == -1) return -1;

    return 1 + Math.max(left, right);
}

boolean isBalanced(Node root) {
    return height(root) != -1;
}
```

5. Diameter of Tree

👉 Diameter = longest path between any 2 nodes

Formula:

$$D = \max(leftHeight + rightHeight, leftDiameter, rightDiameter)$$

Code

```
int diameter(Node root) {
    if (root == null) return 0;

    int leftHeight = height(root.left);
    int rightHeight = height(root.right);

    int leftDia = diameter(root.left);
    int rightDia = diameter(root.right);

    return Math.max(leftHeight + rightHeight, Math.max(leftDia, rightDia));
}
```

6. Maximum Value in Tree

```
int maxValue(Node root) {
    if (root == null) return Integer.MIN_VALUE;

    int left = maxValue(root.left);
    int right = maxValue(root.right);

    return Math.max(root.data, Math.max(left, right));
}
```

7. Level Order Traversal (Revision)

```
void levelOrder(Node root) {
    Queue<Node> q = new LinkedList<>();
    q.add(root);

    while (!q.isEmpty()) {
        Node curr = q.poll();
        System.out.print(curr.data + " ");

        if (curr.left != null) q.add(curr.left);
        if (curr.right != null) q.add(curr.right);
    }
}
```

🚀 Interview Tips (Very Important)

👉 Always remember:

- ✓ Recursion = Tree ka base
- ✓ Height & Diameter = most asked
- ✓ Balanced Tree = tricky question
- ✓ Level order = queue based

🌳 Full Java Program (Input + All Tree Problems)

```java
package Day_41;

import java.util.*;

class Node {
    int data;
    Node left, right;

    Node(int data) {
        this.data = data;
        left = right = null;
    }
}

public class TreeProblems {

    // Build Tree using Level Order Input
    static Node buildTree(Scanner sc) {
        System.out.print("Enter root value: ");
        int val = sc.nextInt();

        if (val == -1) return null;

        Node root = new Node(val);
        Queue<Node> q = new LinkedList<>();
        q.add(root);

        while (!q.isEmpty()) {
            Node curr = q.poll();

            System.out.print("Enter left of " + curr.data + ": ");
            int leftVal = sc.nextInt();
            if (leftVal != -1) {
                curr.left = new Node(leftVal);
                q.add(curr.left);
            }

            System.out.print("Enter right of " + curr.data + ": ");
            int rightVal = sc.nextInt();
            if (rightVal != -1) {
                curr.right = new Node(rightVal);
                q.add(curr.right);
            }
        }

        return root;
    }

    // Height
    static int height(Node root) {
        if (root == null) return 0;
        return 1 + Math.max(height(root.left), height(root.right));
    }

    // Count Nodes
    static int countNodes(Node root) {
        if (root == null) return 0;
        return 1 + countNodes(root.left) + countNodes(root.right);
    }

    // Sum of Nodes
    static int sum(Node root) {
        if (root == null) return 0;
        return root.data + sum(root.left) + sum(root.right);
    }

    // Max Element
```

```java
    static int maxValue(Node root) {
        if (root == null) return Integer.MIN_VALUE;
        return Math.max(root.data, Math.max(maxValue(root.left), maxValue(root.right)));
    }

    // Check Balanced
    static int checkHeight(Node root) {
        if (root == null) return 0;

        int left = checkHeight(root.left);
        int right = checkHeight(root.right);

        if (left == -1 || right == -1) return -1;

        if (Math.abs(left - right) > 1) return -1;

        return 1 + Math.max(left, right);
    }

    static boolean isBalanced(Node root) {
        return checkHeight(root) != -1;
    }

    // Diameter
    static int diameter(Node root) {
        if (root == null) return 0;

        int leftH = height(root.left);
        int rightH = height(root.right);

        int leftD = diameter(root.left);
        int rightD = diameter(root.right);

        return Math.max(leftH + rightH, Math.max(leftD, rightD));
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        Node root = buildTree(sc);

        System.out.println("\n--- OUTPUT ---");
        System.out.println("Height: " + height(root));
        System.out.println("Total Nodes: " + countNodes(root));
        System.out.println("Sum of Nodes: " + sum(root));
        System.out.println("Max Element: " + maxValue(root));
        System.out.println("Is Balanced: " + isBalanced(root));
        System.out.println("Diameter: " + diameter(root));
    }
}

```

🧪 Sample Input (Same Tree jo humne dry run me liya)

Enter root value: 1

Enter left of 1: 2

Enter right of 1: 3

Enter left of 2: 4

Enter right of 2: 5

Enter left of 3: -1

Enter right of 3: -1
Enter left of 4: -1
Enter right of 4: -1
Enter left of 5: -1
Enter right of 5: -1

👉 -1 ka matlab = NULL (no node)

✅ **Expected Output**

--- OUTPUT ---
Height: 3
Total Nodes: 5
Sum of Nodes: 15
Max Element: 5
Is Balanced: true
Diameter: 3

🔥 **Important Notes**

- Ye **level order input** use karta hai (queue based)
- -1 use kiya null node ke liye